

Homework — 2.2.1 Programming Techniques — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. [3] — Completed trace (calls descend to the base case, then the returns unwind):

Call	Value returned
power(2, 4)	16
power(2, 3)	8
power(2, 2)	4
power(2, 1)	2
power(2, 0)	1

- 1 mark: base case row — `power(2, 0)` returns **1** (`exp == 0`).
- 1 mark: intermediate rows correct — 2, 4, 8 (each level returns $2 \times$ the level below).
- 1 mark: final result — `power(2, 4)` returns **16**.

2. [3] — 1 mark per gap: (a) `for`; (b) `next`; (c) `until`. (Distractors: `while`, `endwhile`, `then`.) The `for ... next` loop is count-controlled (fixed number of repeats known in advance); `do ... until` is condition-controlled (repeats until the condition holds, so the number of repeats is not known in advance).

3.

- (a) [2] — 1 mark: first value printed is **8** — the assignment inside `reset()` creates a **local** variable `count` whose scope is the procedure only, so the **global** `count` is never changed. 1 mark: second value printed is **10** — `total` is passed **by value**, so `double` works on a **copy** and the original variable is unchanged. (Both value and justification needed for each mark.)
- (b) [2] — 1 mark: inside `reset()`, declare the variable global — `global count = 0`. 1 mark: pass the parameter by reference — `procedure double(n:byRef)` — so assigning to `n` changes the original variable.

4. [2] — Two correct matches per mark:

Term	Match
Class	A
Object	B
Encapsulation	C
Inheritance	D

Distractors not used: E (a local variable) and F (polymorphism).

5. [2] — 1 mark: `do ... until` tests its condition at the **end** of the loop, so the body always runs **at least once**. 1 mark: the input must be read at least once before it can be tested, so the at-least-once behaviour fits validation exactly; a `while` loop tests first, so it would need the value to exist before the loop / would need duplicated input code before the loop.

Part B (6 marks)

6. [2] — 1 mark: **abstraction** is removing/hiding unnecessary detail to focus on what matters for the problem. 1 mark (any one): makes the problem simpler/more manageable; reduces complexity; produces a general model that can be reused; lets the solver ignore irrelevant detail.

7. [2] — 1 mark: stack = **LIFO** (Last In, First Out). 1 mark: queue = **FIFO** (First In, First Out).

8. [2] — 1 mark: **decomposition** is breaking a complex problem down into smaller sub-problems. 1 mark: each sub-problem is simpler to solve/test separately (and the work can be divided among a team or reused).

Total: 14 + 6 = 20 marks